SAP AMS Transition App — Full Project Handoff Document

> *For 2 interns | 9 days | React + Node.js + PostgreSQL*

---

1. What Is This Project?

You are building a **full-stack web application** called the **SAP AMS Transition App**. It is used when a company switches its SAP support from one vendor to another (called an AMS transition). During this handoff, critical knowledge, contacts, tasks, and team readiness can get lost — this app prevents that by being a central hub for everything.

Think of it as an internal operations tool used by support analysts, project managers, and team leads during a vendor transition.

---

2. What You (The Interns) Have to Build

Out of the 7 total features in the project, **you are responsible for 4**:

FR-2 — Knowledge Base

An internal wiki for SAP support documentation.

- Full-text search across articles
- Rich text editor for writing articles (use **React-Quill**)
- Articles have statuses: `Draft → Under Review → Published`
- Categorize by SAP module (Finance, HR, Supply Chain, etc.) and severity
- Tags and related article links
- Version history (track edits)

FR-3 — Transition Checklist

A task tracker for 3 transition phases.

- Phase 1: Pre-cutover tasks
- Phase 2: Go-live tasks
- Phase 3: Stabilization tasks
- Tasks can have dependencies (Task B unlocks only after Task A is done)
- Assign an owner to each task
- Track completion percentage per phase
- Sign-off documentation when a task is marked complete

FR-4 — Support Team Onboarding *(simplified)*

A training and certification tracker for the incoming support team.

- Assign training modules to team members
- Mark certifications as complete/incomplete
- Track overall readiness per person (% complete)
- *(Skip quizzes and shadow-shifts for now — out of scope for 9 days)*

FR-5 — Key Contact Registry

A directory of who to contact for what issue.

- Map contacts to SAP modules (e.g., "For Finance issues → contact Jane")

- Escalation path (Level 1 → Level 2 → Level 3)
- On-call rotation schedule
- Contact history linked to incidents/modules

---

3. What You Are NOT Building

- FR-1 (Incident Management) — another team's responsibility
- FR-6 (SLA Tracking & Dashboards) — another team's responsibility
- FR-7 (Access Control & Auth) — another team's responsibility
- No SAML/SSO (too complex for 9 days — use JWT instead)
- No Elasticsearch (use PostgreSQL full-text search instead)
- No Docker/Kubernetes (do this last only if time permits)
- No native mobile app
- No SAP OData integration (use mock data)

---

4. Tech Stack

Layer	Technology	Why
Frontend	React.js 18+	As per project doc
UI Components	Material-UI (MUI) v5	As per project doc
Rich Text Editor	React-Quill	Best React rich text editor
Backend	Node.js + Express.js	As per project doc, same language as frontend
Database	PostgreSQL 14+	As per project doc
Auth	JWT (JSON Web Tokens)	Simpler than SAML for 9 days
HTTP Client	Axios	For frontend API calls
ORM/Query	pg (node-postgres)	Direct SQL, no ORM overhead
Environment	dotenv	Manage secrets

---

5. Folder Structure (VS Code)

```
sap-ams-app/
│
├── client/                          # React Frontend
│   ├── public/
│   │   └── index.html
│   ├── src/
│   │   ├── api/                     # Axios API call functions
│   │   │   ├── knowledge.js
│   │   │   ├── checklist.js
│   │   │   ├── onboarding.js
│   │   │   └── contacts.js
│   │   ├── components/              # Reusable UI components
│   │   │   ├── Layout/
│   │   │   │   ├── Sidebar.jsx
│   │   │   │   ├── Topbar.jsx
│   │   │   │   └── Layout.jsx
│   │   │   ├── common/
│   │   │   │   ├── LoadingSpinner.jsx
│   │   │   │   ├── ConfirmDialog.jsx
│   │   │   │   └── StatusChip.jsx
│   │   ├── pages/                   # One folder per feature
│   │   │   ├── KnowledgeBase/
│   │   │   │   ├── KnowledgeList.jsx
│   │   │   │   ├── KnowledgeDetail.jsx
│   │   │   │   └── KnowledgeEditor.jsx
│   │   │   ├── Checklist/
│   │   │   │   ├── ChecklistDashboard.jsx
│   │   │   │   └── TaskCard.jsx
│   │   │   ├── Onboarding/
│   │   │   │   ├── OnboardingDashboard.jsx
│   │   │   │   └── MemberCard.jsx
│   │   │   ├── Contacts/
│   │   │   │   ├── ContactList.jsx
│   │   │   │   └── ContactDetail.jsx
│   │   │   └── Auth/
│   │   │       ├── Login.jsx
│   │   │       └── Register.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx      # Global auth state
│   │   ├── hooks/
│   │   │   └── useAuth.js
│   │   ├── utils/
│   │   │   └── helpers.js
│   │   ├── App.jsx
│   │   └── index.js
│   └── package.json
│
├── server/                          # Node.js + Express Backend
│   ├── config/
│   │   └── db.js                    # PostgreSQL connection
│   ├── middleware/
│   │   ├── auth.js                  # JWT verification middleware
│   │   └── errorHandler.js
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── knowledge.routes.js
│   │   ├── checklist.routes.js
│   │   ├── onboarding.routes.js
│   │   └── contacts.routes.js
│   ├── controllers/
```

```
│   │   ├── auth.controller.js
│   │   ├── knowledge.controller.js
│   │   ├── checklist.controller.js
│   │   ├── onboarding.controller.js
│   │   └── contacts.controller.js
│   ├── db/
│   │   └── schema.sql              # Full DB schema
│   ├── .env                        # Environment variables (never commit this)
│   ├── .env.example                # Template for others
│   ├── index.js                    # Entry point
│   └── package.json
│
└── README.md
```

---

6. Database Schema (PostgreSQL)

```
-- USERS
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL CHECK (role IN ('admin', 'analyst', 'manager', 'viewer')),
  department VARCHAR(100),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- KNOWLEDGE ARTICLES
CREATE TABLE knowledge_articles (
  article_id SERIAL PRIMARY KEY,
  title VARCHAR(500) NOT NULL,
  content TEXT NOT NULL,
  module VARCHAR(100),          -- Finance, HR, Supply Chain, etc.
  severity VARCHAR(50) CHECK (severity IN ('critical', 'high', 'medium', 'low')),
  status VARCHAR(50) DEFAULT 'draft' CHECK (status IN ('draft', 'under_review', 'published')),
  tags TEXT[],                  -- PostgreSQL array of tags
  created_by INTEGER REFERENCES users(user_id),
  approved_by INTEGER REFERENCES users(user_id),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  search_vector TSVECTOR         -- For full-text search
);

-- Auto-update search vector
CREATE INDEX knowledge_search_idx ON knowledge_articles USING GIN(search_vector);
CREATE OR REPLACE FUNCTION update_search_vector() RETURNS TRIGGER AS $$
BEGIN
  NEW.search_vector := to_tsvector('english', NEW.title || ' ' || NEW.content);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER knowledge_search_trigger
  BEFORE INSERT OR UPDATE ON knowledge_articles
  FOR EACH ROW EXECUTE FUNCTION update_search_vector();

-- RELATED ARTICLES (many-to-many)
CREATE TABLE related_articles (
  article_id INTEGER REFERENCES knowledge_articles(article_id) ON DELETE CASCADE,
  related_id INTEGER REFERENCES knowledge_articles(article_id) ON DELETE CASCADE,
  PRIMARY KEY (article_id, related_id)
);

-- TRANSITION TASKS (Checklist)
CREATE TABLE transition_tasks (
  task_id SERIAL PRIMARY KEY,
  title VARCHAR(500) NOT NULL,
  description TEXT,
  phase VARCHAR(50) NOT NULL CHECK (phase IN ('pre_cutover', 'go_live', 'stabilization')),
  status VARCHAR(50) DEFAULT 'pending' CHECK (status IN ('pending', 'in_progress', 'completed', 'blocked')),
  due_date DATE,
  assigned_to INTEGER REFERENCES users(user_id),
  completed_at TIMESTAMP,
  signed_off_by INTEGER REFERENCES users(user_id),
  sign_off_notes TEXT,
```

```
  created_at TIMESTAMP DEFAULT NOW()
);

-- TASK DEPENDENCIES
CREATE TABLE task_dependencies (
  task_id INTEGER REFERENCES transition_tasks(task_id) ON DELETE CASCADE,
  depends_on_id INTEGER REFERENCES transition_tasks(task_id) ON DELETE CASCADE,
  PRIMARY KEY (task_id, depends_on_id)
);

-- ONBOARDING MEMBERS
CREATE TABLE onboarding_members (
  member_id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(user_id),
  role_assigned VARCHAR(255),
  start_date DATE,
  target_completion_date DATE,
  overall_status VARCHAR(50) DEFAULT 'in_progress' CHECK (overall_status IN ('not_started', 'in_progress', 'certified')),
  created_at TIMESTAMP DEFAULT NOW()
);

-- TRAINING MODULES
CREATE TABLE training_modules (
  module_id SERIAL PRIMARY KEY,
  title VARCHAR(500) NOT NULL,
  description TEXT,
  sap_module VARCHAR(100),
  is_required BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);

-- TRAINING ASSIGNMENTS (which member is assigned which module)
CREATE TABLE training_assignments (
  assignment_id SERIAL PRIMARY KEY,
  member_id INTEGER REFERENCES onboarding_members(member_id) ON DELETE CASCADE,
  module_id INTEGER REFERENCES training_modules(module_id),
  status VARCHAR(50) DEFAULT 'not_started' CHECK (status IN ('not_started', 'in_progress', 'completed')),
  completed_at TIMESTAMP,
  notes TEXT
);

-- KEY CONTACTS
CREATE TABLE key_contacts (
  contact_id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(user_id),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  phone VARCHAR(50),
  role VARCHAR(255),
  sap_module VARCHAR(100),
  escalation_level INTEGER CHECK (escalation_level IN (1, 2, 3)),
  is_on_call BOOLEAN DEFAULT FALSE,
  on_call_start TIME,
  on_call_end TIME,
  notes TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- ESCALATION PATHS
CREATE TABLE escalation_paths (
```

```
  path_id SERIAL PRIMARY KEY,
  sap_module VARCHAR(100) NOT NULL,
  level_1_contact INTEGER REFERENCES key_contacts(contact_id),
  level_2_contact INTEGER REFERENCES key_contacts(contact_id),
  level_3_contact INTEGER REFERENCES key_contacts(contact_id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

---

7. API Endpoints

Auth

```
POST   /api/auth/register
POST   /api/auth/login
GET    /api/auth/me
```

Knowledge Base

```
GET    /api/knowledge                 # List all (with search ?q=keyword)
GET    /api/knowledge/:id             # Get single article
POST   /api/knowledge                 # Create article
PUT    /api/knowledge/:id             # Update article
DELETE /api/knowledge/:id             # Delete article
PATCH  /api/knowledge/:id/status      # Change status (draft→review→published)
GET    /api/knowledge/:id/related     # Get related articles
```

Transition Checklist

```
GET    /api/checklist                 # Get all tasks (optional ?phase=pre_cutover)
GET    /api/checklist/:id             # Get single task
POST   /api/checklist                 # Create task
PUT    /api/checklist/:id             # Update task
PATCH  /api/checklist/:id/complete    # Mark complete + sign-off
GET    /api/checklist/progress        # Get % complete per phase
```

Onboarding

```
GET    /api/onboarding                # List all members + progress
GET    /api/onboarding/:id            # Single member detail
POST   /api/onboarding                # Add member
PUT    /api/onboarding/:id/assignment # Update training assignment status
GET    /api/onboarding/modules        # List all training modules
POST   /api/onboarding/modules        # Create training module
```

Key Contacts

```
GET    /api/contacts                  # List all contacts
GET    /api/contacts/:id              # Single contact
POST   /api/contacts                  # Create contact
PUT    /api/contacts/:id              # Update contact
DELETE /api/contacts/:id              # Delete contact
GET    /api/contacts/escalation/:module  # Get escalation path for a module
```

---

8. Key npm Packages to Install

Server (`/server`)

```
npm install express pg bcryptjs jsonwebtoken cors dotenv
npm install --save-dev nodemon
```

Client (`/client`)

```
npx create-react-app .
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material
npm install react-router-dom axios react-quill
```

---

9. Environment Variables (`server/.env`)

```
PORT=5000
DATABASE_URL=postgresql://postgres:yourpassword@localhost:5432/sap_ams_db
JWT_SECRET=your_super_secret_jwt_key_here
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:3000
```

---

10. 9-Day Build Plan

Day	Person A (Backend)	Person B (Frontend)
1	Setup repo, DB schema, Express boilerplate, .env	React app setup, MUI theme, Sidebar + Topbar layout
2	Auth API (register, login, JWT middleware)	Login page, protected routes, AuthContext
3	Knowledge Base CRUD + search API	Knowledge Base list page + search bar
4	Knowledge Base status workflow API	Article editor with React-Quill + detail view
5	Transition Checklist API (phases, tasks, dependencies)	Checklist dashboard with phase tabs + task cards
6	Key Contact Registry API	Contact list + escalation path UI
7	Onboarding API (members + training assignments)	Onboarding dashboard + connect all APIs
8	Seed dummy data, fix backend bugs	Polish UI, fix broken flows, error states
9	Deploy to Render.com (free), write README	Final demo prep, test all flows end-to-end

---

11. Deployment (Day 9 — Free & Fast)

Use **Render.com** — it's free and deploys Node + PostgreSQL easily.

1. Push code to GitHub
2. Create a PostgreSQL database on Render (free tier)
3. Deploy backend as a Web Service (set env variables in dashboard)
4. Deploy frontend as a Static Site (set `REACT_APP_API_URL` to backend URL)

---

12. Pages to Build in React

Page	Route	Feature
Login	`/login`	Auth
Dashboard/Home	`/`	Overview stats
Knowledge List	`/knowledge`	FR-2
Knowledge Detail	`/knowledge/:id`	FR-2
Knowledge Editor	`/knowledge/new` or `/knowledge/:id/edit`	FR-2
Checklist	`/checklist`	FR-3

Page	Route	Feature
Onboarding	`/onboarding`	FR-4
Contacts	`/contacts`	FR-5

---

13. Important Decisions & Shortcuts Taken

What the doc says	What you're actually building	Why
Elasticsearch for search	PostgreSQL `tsvector` full-text search	Saves 2 days of setup
SAML/SSO auth	JWT auth	Too complex for 9 days
Docker + Kubernetes	Plain Node.js, deploy to Render	Not needed for internship demo
Quizzes in onboarding	Skipped	Out of scope for 9 days
SAP OData integration	Mock data	No SAP access

---

14. How to Start Right Now (Day 1 Commands)

```
# 1. Create project folder
mkdir sap-ams-app && cd sap-ams-app
git init

# 2. Create server
mkdir server && cd server
npm init -y
npm install express pg bcryptjs jsonwebtoken cors dotenv
npm install --save-dev nodemon

# 3. Create client
cd ..
npx create-react-app client
cd client
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material react-router-dom axios react-quill

# 4. Create PostgreSQL database
createdb sap_ams_db
# Then run the schema.sql file in /server/db/schema.sql
psql -d sap_ams_db -f server/db/schema.sql
```

Built for: SAP AMS Transition App Internship Project | NTT DATA Business Solutions Timeline: 9 days | Team: 2 interns | Stack: React + Node.js + PostgreSQL